

SAGE (Semantic Analysis and Generation Engine): Architecture and Design of a Multi-Source Retrieval-Augmented Question Answering System

Srujan Zanjali
Independent Project
Email: srujan.zanjali@infocepts.com

Abstract—Retrieval-augmented generation (RAG) systems are usually built and evaluated against a single, homogeneous corpus of text documents. Real knowledge work rarely looks like that: a person researching a topic collects a project website, a set of PDFs, a recorded lecture, and a code repository, and expects to ask one conversational question across all of them. We present SAGE (Semantic Analysis and Generation Engine), a system that ingests four structurally different source types — crawled websites, PDF documents, YouTube video transcripts, and GitHub code repositories — through a shared ingest-chunk-embed-index contract, and answers questions over them with a single retrieval-reranking-generation pipeline. We describe SAGE’s four modality-specific extractors and chunking strategies; a deterministic, non-generative source-overview chunk mechanism that synthesizes a citable summary of each ingested source from its own structure and statistics rather than from a second LLM call; a hybrid semantic-lexical reranking formula combined with a diversity floor that guarantees fair citation representation when a query spans multiple sources; and a confidence heuristic that gates a grounded, citation-only answer mode against an exploratory mode. We report on the system’s real test suite (95 tests across 12 files) as validation evidence, describe a concrete design defect — silent starvation of one source’s evidence in multi-source queries — that was discovered during this project’s own testing and fixed by the diversity floor, and discuss the system’s limitations honestly. SAGE’s contribution is not a new model but a concrete, checkable set of engineering decisions for building grounded QA over heterogeneous, real-world knowledgebases.

Index Terms—retrieval-augmented generation, multi-source retrieval, question answering, reranking, chunking, confidence estimation

I. Introduction

Retrieval-augmented generation grounds a language model’s answer in text retrieved from an external corpus, rather than relying solely on parameters learned at training time [1]. Most published RAG systems, and most open-source RAG frameworks, are designed and evaluated against a single corpus of textual documents: a Wikipedia dump, a set of PDFs, or a company’s internal wiki. In practice, the knowledge a person or team wants to query is rarely that uniform. A student researching a topic has a lecture video, a set of assigned PDFs, and a related GitHub repository open simultaneously; a developer onboarding onto a project has the project’s marketing website, its documentation PDF, a walkthrough video, and the codebase itself. Asking a single conversational

question that should be answered by whichever of these sources actually contains the relevant evidence — and that returns a citation naming exactly where the evidence came from — requires a system that treats extraction, chunking, retrieval, and citation as a single contract across source types that otherwise share almost nothing in their native structure.

SAGE (Semantic Analysis and Generation Engine) is such a system. It ingests four source types — crawled multi-page websites, PDF documents, YouTube video transcripts, and GitHub code repositories — into a common vector index, and answers questions against one, several, or all of a user’s ingested sources through a single retrieval-augmented pipeline that performs semantic-lexical reranking, enforces per-source citation diversity on combined queries, gates its own confidence, and streams incremental answers with citations attached as they are produced. This paper describes SAGE’s architecture and the specific, checkable engineering decisions behind it, rather than proposing a new retrieval or generation model. Three of these decisions are, to our knowledge, not widely documented elsewhere in this combination:

- **Deterministic source-overview chunks.** For each ingested source, SAGE synthesizes one additional chunk — placed first in that source’s chunk sequence — built purely from the source’s own statistics, structure, and excerpts (page titles, transcript segment timestamps, a repository’s on-disk directory tree), with no LLM call. Its only purpose is to make broad, non-specific questions such as “what is this about” retrievable, since such questions have poor lexical and semantic overlap with any single content chunk.
- **A reranking-plus-diversity-floor combination** discovered as a bug fix. Ranking retrieved chunks purely by a blended semantic-lexical score, across a query that spans multiple user-selected sources, can silently starve one source of any citation at all if its best-matching chunk scores lower than another source’s several good chunks. This was observed during development as a real defect, not designed in advance, and the fix — reserving each combined source’s single best chunk before filling remaining slots by score —

is now a permanent feature of the pipeline.

- A confidence heuristic that gates refusal, not just a display badge. SAGE computes a single scalar confidence score from retrieval evidence alone (no second LLM call to self-assess) and uses it to decide, before generation, whether Grounded Mode should answer at all or refuse and ask a clarifying follow-up question — with an explicit, narrower exemption for video sources, which degrade to a warning rather than a hard refusal.

The remainder of this paper is organized as follows. Section II situates SAGE against the retrieval, reranking, and chunking literature it builds on. Section III gives the system’s end-to-end architecture. Section IV describes the four ingestion pipelines. Section V details the source-overview mechanism. Section VI gives the exact reranking and diversity-floor formulas. Section VII describes confidence scoring and answer-mode gating. Section VIII covers query rewriting, streaming, and memory. Section IX briefly discusses security posture. Section X reports validation evidence and honest limitations, and Section XI concludes.

II. Related Work

Dense retrieval and RAG. Neural retrieval replaced sparse lexical matching with learned dense representations, from early semantic-matching models such as DSSM [27] through dual-encoder retrievers such as DPR [2], which SAGE’s dense-retrieval-plus-reranking design descends from directly. Retrieval-augmented generation [1] formalized conditioning a generator on retrieved passages rather than parametric memory alone; Fusion-in-Decoder [3] showed that a generator can effectively marginalize over many retrieved passages at once, which is architecturally close to how SAGE assembles a single multi-chunk context window per answer. Gao et al. [4] survey the broader RAG design space that SAGE occupies one point within.

Sentence embeddings. SAGE’s embedding model is a distilled, sentence-level bi-encoder in the SBERT lineage [5], specifically a MiniLM-based distillation [6], [7] released as the all-MiniLM-L6-v2 sentence-transformers checkpoint [8]. This model trades some retrieval accuracy against much larger encoders for the low latency needed in an interactive, streaming QA product.

Reranking and hybrid retrieval. Cross-encoder rerankers [10] and late-interaction models such as ColBERT [11] improve on bi-encoder-only retrieval by scoring query-passage pairs jointly, at additional inference cost. SAGE instead uses a cheap, closed-form hybrid of its bi-encoder’s semantic score with a lexical term-recall score, in the spirit of lexical-dense combination approaches such as CLEAR [12] and classical BM25 [9], and of rank-fusion techniques such as Reciprocal Rank Fusion [13] — without the latency cost of a second neural pass over every candidate.

Chunking. Fixed-size chunking with overlap remains the dominant practical strategy in production RAG systems,

but recent work has begun to study its tradeoffs explicitly, including chunk-size sensitivity in domain-specific documents [14], embedding chunks with surrounding long-context information retained (“late chunking”) [15], and comparative evaluation of chunking strategies [16]. SAGE uses a sentence-aware fixed-budget chunker for its three textual modalities and a distinct character-band chunker for video transcripts, discussed in Section IV.

Modality-specific retrieval. Video question answering over transcripts and timestamps has its own benchmark and method literature [17], [18]; code search and retrieval-augmented code completion has developed specialized retrieval strategies around repository structure [19], [20], [21]; and multi-document retrieval and reading has been studied as an end-to-end trainable problem [22]. SAGE does not train a specialized retriever for any one modality; instead it normalizes all four modalities into one embedding space and instead specializes the overview-chunk generation and context-budgeting logic per modality, described in Sections V–VI.

Confidence and abstention. Selective prediction under distribution shift [23], self-reported model confidence [24], and hallucination in generation [25] motivate SAGE’s decision to gate refusal on a retrieval-derived confidence score rather than trust the generator’s own certainty; retrieval augmentation has independently been shown to reduce hallucination in open-ended generation [26].

Security is not this paper’s subject, but SAGE’s ingestion of arbitrary user-submitted URLs and its context window built from untrusted retrieved text both required standard mitigations: server-side request forgery defenses [28] and indirect prompt-injection guarding [29], covered briefly in Section IX. Consumer products such as NotebookLM [30] and general-purpose RAG frameworks such as LangChain [31] and LlamaIndex [32] address an overlapping problem at a different layer (a polished single-user product, and a generic composability library, respectively); SAGE is neither, and comparison to either is not this paper’s focus.

III. System Architecture

SAGE is a FastAPI backend with a React web client and a Chrome side-panel extension, backed by ChromaDB as the vector store (HNSW index, cosine space) and Supabase as the relational store of record for sources, knowledgebases, chunk metadata, and job status. Figure 1 shows the two halves of the system: an ingestion half, in which each of the four source types is extracted by its own modality-specific extractor but produces a common chunk representation, and a query half, in which every question — regardless of which source types it targets — passes through one retrieval, reranking, confidence, and generation pipeline.

A source is one ingested website, PDF, video, or repository; sources are grouped into knowledgebases, and a query can target a single source, an entire knowledgebase,

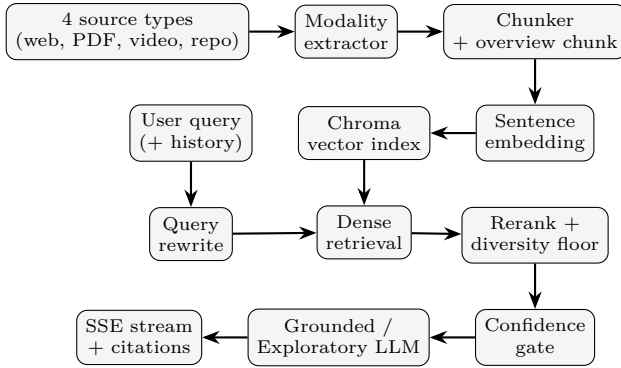


Fig. 1. SAGE end-to-end architecture: a shared ingest-chunk-embed-index contract across four source types feeds a single retrieve-rerank-gate-generate query pipeline.

or an explicit list of combined `source_ids` spanning one or more knowledgebases. Every chunk, regardless of modality, carries a common metadata schema: a knowledgebase and source identifier, a source type, a chunk index, and a content-addressed chunk identifier computed as the truncated SHA-256 hash of the knowledgebase id, source id, chunk index, and chunk text, which makes re-ingestion of unchanged content naturally idempotent at the vector-store upsert layer.

A. Persistence and Metadata

Two stores back every source, kept in intentional sync rather than merged into one system. ChromaDB holds only what retrieval needs: chunk text, its embedding vector, and a flat metadata dictionary (Chroma rejects None-valued metadata fields outright, so the upsert layer strips any unset field before writing rather than passing it through). Supabase holds the relational structure retrieval does not need but the product does: knowledgebase and source records, human-facing titles and URLs, ingestion timestamps, and — for the two source types that ingest asynchronously — job status rows polled through `/jobs/{job_id}`. A source’s `chunk_kind` metadata field is the mechanism that lets one shared context-assembly routine (Section VI) special-case exactly two chunk categories — ordinary content chunks and the source-overview chunk described next — without needing to know which of the four modalities produced either one. The background job tracker itself is a plain in-process dictionary with no persistence or expiry, a deliberate prototype simplification we return to as a limitation in Section X.

Table I lists the system’s REST surface.

Two implementation details are worth surfacing because they shape the rest of this paper. First, GitHub ingestion has no synchronous endpoint at all — cloning, scanning, and chunking a repository is always dispatched as a background job polled via `/jobs/{job_id}`, unlike website and PDF ingestion, which are synchronous. Second, video ingestion is exposed as two independent, explicitly user-triggered endpoints rather than one endpoint with an

TABLE I
SAGE API surface (`/api/v1` prefix)

Method	Path	Purpose
GET	<code>/health</code>	liveness check
POST	<code>/qa/ask</code>	synchronous QA
POST	<code>/qa/ask/stream</code>	SSE streaming QA
POST	<code>/ingest/website</code>	synchronous multi-page crawl
POST	<code>/ingest/pdf</code>	synchronous PDF upload
POST	<code>/ingest/video</code>	async, transcript-API path
POST	<code>/ingest/video-auto-transcribe</code>	async, Whisper fallback
POST	<code>/jobs/github-ingest</code>	async, only GitHub ingest path
GET	<code>/jobs/{job_id}</code>	poll async job status
GET	<code>/sources,</code> <code>/sources/grouped</code>	list sources
GET	<code>/sources/{id}/summary</code>	deterministic overview text
DELETE	<code>/sources/{id}</code>	delete source, its vectors
POST	<code>/sources/website/recrawl</code>	async re-crawl of a source

automatic fallback: `/ingest/video` attempts the YouTube transcript API and fails with a distinguishable error if no transcript exists; the client then offers the user a separate, explicit “transcribe automatically” action that calls `/ingest/video-auto-transcribe`, which runs `yt-dlp` audio extraction followed by local Whisper transcription. There is no server-side chaining between the two.

IV. Multi-Source Ingestion

Each modality has its own extractor, but every extractor ends at the same handoff point: a list of `TextChunk` objects with metadata, ready for embedding and upsert into the vector store. Table II summarizes the four pipelines; the remainder of this section describes each in turn.

A. Website

The production website path is a headless Playwright crawl (`website_crawler.py`), not the simpler single-page requests+BeautifulSoup fetcher that also exists in the codebase for legacy use. The crawler performs a breadth-first traversal restricted to the starting page’s domain, blocks image/font/media sub-resource loading to reduce crawl cost, and re-validates every followed link — including mid-crawl redirect targets, not just the entry URL — against a public-address check before navigating, discussed further in Section IX. Extracted page text is normalized and passed to the shared sentence-aware chunker described below. Websites are deduplicated on a normalized URL (stripped `www.`, default ports, and tracking query parameters such as `utm_*`, `gclid`, `fbclid`), so re-submitting the same page under a decorated URL reuses the existing source rather than re-ingesting it.

B. PDF

PDF upload is validated by a byte-signature check (`%PDF` magic bytes), independent of the declared HTTP

TABLE II
Per-modality ingestion summary

Modality	Extraction method	Chunking unit	Chunk size band	Notable caps
Website	Playwright multi-page crawl (BFS)	sentence-aware, char budget	1200 chars / 180 overlap	same-domain only; SSRF-checked per hop
PDF	PyMuPDF (fitz) per-page text	sentence-aware, char budget	1200 chars / 180 overlap	25MB upload; born-digital text only
Video	YouTube transcript API / Whisper (faster-whisper)	segment accumulation	900 target / 700–1000 chars	900s duration, 100MB audio (auto path)
GitHub repo	git clone –depth 1 (ZIP fallback)	line-range accumulation	40–120 lines	300 files, 1200 chunks, 2,000,000 chars, 250 KB/file

content type, and capped at 25MB. Extraction uses PyMuPDF’s per-page `get_text("text")`; if the concatenated text across all pages is empty (a scanned, image-only PDF, for example), ingestion fails with an explicit “OCR is not implemented” error rather than silently indexing nothing. A minimum extracted-length floor further rejects near-empty documents. Deduplication is content-addressed: the SHA-256 hash of the uploaded bytes is the canonical source key, so byte-identical re-uploads under a different filename are recognized as the same source.

C. Video

Given a YouTube URL, the primary path calls the YouTube transcript API to obtain caption segments with timestamps. Raw caption segments are far shorter than a useful chunk, so video transcripts use a dedicated chunker distinct from the shared sentence-aware chunker: segments are greedily accumulated toward a 900-character target within a 700–1000-character band, and any trailing chunk left under 260 characters after the pass is merged into its predecessor rather than kept as an undersized final chunk. When no transcript is available, the user can explicitly trigger the fallback path, which downloads audio with `yt-dlp` and transcribes it locally with `faster-whisper` (default model size `tiny`, beam size `1`, voice-activity-detection filtering enabled, with an `int8` → `int8_float16` → `float32` compute-type fallback chain depending on host capability), subject to a 900-second duration cap and 100MB audio cap validated from video metadata before download begins. Both paths preserve per-chunk start/end timestamps in metadata, which are surfaced later as `[MM:SS–MM:SS]` citation labels.

D. GitHub Repository

A repository is fetched with a two-tier strategy: a shallow git clone `–depth 1` first, falling back on failure to downloading and safely extracting a GitHub archive ZIP (trying the requested branch, then `main`, then `master`), with explicit path-traversal (“zip-slip”) protection during extraction. The cloned tree is then scanned file by file, skipping binary files (detected by a null byte in the first 8000 bytes), lockfiles, and common non-source directories (`.git`, `node_modules`, `dist`, `build`, `.venv`, and similar), and is bounded by five independent caps — 300 indexed files,

1200 total chunks, 2,000,000 total characters, 250 KB per file, and 100,000 lines per file — so that ingesting a very large monorepo degrades to a partial, capped index rather than failing or exhausting resources. Chunking is line-range based rather than character-budget based: chunks target 40–120 lines, with a trailing under-40-line remainder merged into the previous chunk, which keeps code chunks aligned to something closer to function- or block-scale rather than an arbitrary character cut. Repositories are deduplicated by their canonical owner/repo URL regardless of query string or trailing slash.

E. Shared Chunker

Website and PDF text (and, with a distinct parameterization, video transcripts) pass through a common sentence-aware chunker. Paragraphs are split on blank lines, then sentences within a paragraph on a locale-aware sentence-boundary regular expression; sentences are accumulated into a chunk until adding the next sentence would exceed a 1200-character budget, at which point the chunk closes and a new one opens, seeded with the last 180 characters of the chunk that just closed so that a fact split across a chunk boundary is not orphaned from its immediate context. This is the chunk-size/overlap band studied for sensitivity elsewhere [14]; SAGE fixes it as a global default and does not vary it adaptively by document type, a limitation discussed in Section X.

F. Deduplication and Idempotency

Because a user may submit the same source more than once — the same page under a tracking-decorated URL, the same PDF re-uploaded under a new filename, the same YouTube video pasted twice, or the same repository re-ingested to pick up new commits — every modality defines its own notion of canonical identity, checked before any extraction work begins. Websites are deduplicated on a normalized URL: the scheme is fixed, a leading `www.` and default port are stripped, and known tracking query parameters (`utm_source`, `utm_medium`, `gclid`, `fbclid`, and similar) are removed before the remaining query parameters are sorted, so two URLs that differ only in tracking decoration or parameter order resolve to the same source. PDFs are deduplicated on the SHA-256 hash of the uploaded bytes themselves, used directly

as the source’s canonical reference (pdf:{hash}), which is stronger than filename- or title-based matching since it is invariant to how the file was renamed or re-described. Videos are deduplicated on the canonical YouTube video identifier extracted from any of its equivalent URL forms (watch?v=,youtu.be/, or /shorts/). GitHub repositories are deduplicated on owner/repo alone, independent of trailing slashes or query strings, so re-ingesting the same repository under a slightly different URL updates rather than duplicates it.

This same content-addressing principle is applied one level down, at the chunk itself: every chunk’s identifier is the truncated SHA-256 hash of its knowledgebase id, source id, index, and text, so re-ingesting a source whose content has not changed produces byte-identical chunk ids and is naturally idempotent at Chroma’s upsert layer — no separate “has this changed” check is needed, because unchanged content simply overwrites itself with an identical vector and metadata record. A website’s explicit recrawl path, notably, does not rely on this idempotency: it deletes the entire prior knowledgebase’s chunk rows and vectors unconditionally and re-ingests from a fresh crawl, rather than diffing old and new chunk sets — a simpler, correctness-favoring design at the cost of re-embedding unchanged pages on every recrawl, which we note as a possible efficiency improvement in Section X.

V. Source-Overview Chunks

A short, non-specific question — “what is this website about,” “summarize this video,” “what does this repo do” — has poor term overlap and often poor semantic similarity with any single 1200-character content chunk, because the answer to such a question lives in the source’s structure as a whole, not in any one passage of it. Rather than ask a second LLM call to summarize each source at ingestion time (which would add cost, latency, and yet another place for hallucination to enter the system before a user ever asks a question), SAGE constructs one additional, deterministic overview chunk per source from the source’s own statistics and excerpts, and always inserts it as chunk index 0. This is, to our knowledge, the most original engineering decision in the system, and the one we give the most depth here.

Every overview chunk shares a small set of primitives. A crude term-frequency pass (`_top_terms`) tokenizes all of a source’s text, lowercases it, removes a hand-built stopwords list that includes generic words specific to the domain (“video,” “website,” “chunk”), and keeps the six most frequent remaining terms as a topic line — deliberately term-frequency only, with no TF-IDF weighting and no embedding step, since the overview’s job is to be retrievable by lexical overlap with a vague query, not to be a precise summary. A first/last excerpt helper takes the first and last few structural units of a source (pages, transcript segments, or repository files) to give the chunk a sense of beginning and end. Every overview

chunk’s underlying text is prefixed with an internal marker (for example `__SAGE_VIDEO_OVERVIEW__`) that is stripped back out whenever the chunk is displayed to a user (for instance via the `/sources/{id}/summary` endpoint), and is exempted from the per-chunk truncation limit applied to ordinary content chunks during context assembly (Section VI), so the overview is never cut short inside the model’s context window.

Website. The website overview draws on the crawl’s title, canonical start URL, the count of pages attempted versus successfully crawled, the total chunk count, up to five title — url rows for crawled pages, first/last excerpts from two crawled pages, and top terms pooled across every crawled page’s text.

PDF. The PDF overview draws on filename, page count, total extracted character count, and chunk count, plus a lightweight heading detector applied only to the first three pages: a line under 90 characters qualifies as a heading if it is fully uppercase, begins with #, or matches a simple title-case pattern, capped at six detected headings. First/last excerpts use one page each; top terms are pooled across all pages.

Video. The video overview draws on title, video identifier, duration, and transcript origin (YouTube-native captions versus Whisper-transcribed), and reuses the already-chunked 700–1000-character transcript chunks — not raw caption lines — as its structural unit, rendering `[MM:SS–MM:SS]` text rows for the first and last three chunks and pooling top terms across all transcript chunks. Its metadata carries a synthetic “Full video” timestamp label spanning the whole duration, distinguishing it from ordinary timestamped segment chunks.

GitHub repository. The repository overview is qualitatively different from the other three: it is filesystem-introspective, re-walking the already-cloned repository directory on disk rather than deriving everything from already-produced chunk metadata. It lists top-level directories (excluding the same skip-list used during scanning), checks for the literal on-disk presence of roughly ten category-tagged conventions — a README, a requirements/dependency manifest, setup/configuration files, a test directory, likely main-entry-point files, routes/controllers, models/schemas, and services — and renders a depth-two ASCII directory tree (the first 15 top-level entries, each with up to five children, using box-drawing characters), alongside the union of file extensions observed and counts of indexed versus skipped files. Because it depends on the filesystem rather than only on already-extracted text, the repository overview is inserted into the chunk sequence through a slightly different code path than the other three modalities’ overview chunks, though it occupies the same chunk-index-0 position and carries the same `chunk_kind` metadata contract. We consider this the strongest single original artifact in SAGE: it is a structurally rich, directly verifiable summary of a codebase that requires no model inference at all, generated

in the same pass as ordinary chunk extraction.

VI. Retrieval, Reranking, and the Diversity Floor

A. Retrieval

A query (after the rewriting step described in Section VIII) is embedded with the same all-MiniLM-L6-v2 sentence encoder used at ingestion, with L2-normalized output vectors, and searched against Chroma’s HNSW index configured for cosine distance. The candidate pool size is not fixed: for a GitHub-only query, the pool is $\min(2k, 8)$ where k is the requested top- k (default 6, capped at 6 for GitHub regardless of the client’s request); for other single-source queries it is $2k$ uncapped; and when a query explicitly combines $n \geq 2$ source_ids, the pool is widened further to $2k \cdot n$, capped at 40, specifically so that the diversity floor below has enough real per-source candidates to reserve from, rather than reserving from an already-truncated top- k list dominated by one source.

B. Reranking

Each retrieved chunk’s raw semantic score is $s = \text{clip}_{[0,1]}(1 - d)$, where d is Chroma’s returned cosine distance. This is combined with a lexical term-recall score $\ell = |Q \cap T|/|Q|$, where Q is the set of lowercased query tokens of length ≥ 3 (alphanumeric plus underscore, minus a small stopword list) and T is the corresponding token set of the chunk text — i.e., the fraction of query terms that literally appear in the chunk, not a symmetric Jaccard overlap. The final rerank score is the fixed linear blend

$$\text{score} = 0.78s + 0.22\ell, \quad (1)$$

rounded to four decimal places. This gives semantic similarity the dominant weight while still rewarding exact keyword matches — useful for proper nouns, error codes, or API names that a bi-encoder alone may not distinguish sharply. Unlike a cross-encoder reranker [10] or ColBERT-style late interaction [11], Equation 1 requires no additional forward pass through a neural network at query time; both terms are available from the retrieval step and a cheap token-set intersection.

C. The Diversity Floor

Reranking chunks purely by Equation 1 and truncating to top- k works well for a single-source query, but has a specific failure mode when a query explicitly combines multiple sources: if source A happens to contain several moderately good chunks and source B contains one excellent chunk, a pure score-sort can push B ’s only relevant chunk outside the top- k cutoff entirely, so the final answer cites only A and silently omits any evidence from B even though B was one of the sources the user explicitly asked about. This was discovered as a real defect during this project’s own manual testing of combined-source queries, not designed for in advance, and is now handled by a diversity floor (Algorithm 1), which activates only when a request explicitly names two or more combined source_ids

Algorithm 1 Diversity-floor selection

Require: ranked chunks C sorted by score desc.; requested k ; combined source ids S , $|S| \geq 2$

- 1: $R \leftarrow \emptyset$ $\triangleright$ reserved slots, one per source
- 2: for $c \in C$ do
- 3: if $c.\text{source} \notin \{r.\text{source} : r \in R\}$ and $|R| < k$ then
- 4: $R \leftarrow R \cup \{c\}$
- 5: end if
- 6: end for
- 7: $F \leftarrow$ next highest-scored chunks in $C \setminus R$, filling until $|R \cup F| = k$
- 8: return $\text{sort_by_score_desc}(R \cup F)$

— a single source, or a whole knowledgebase reference, is unaffected and uses plain score ranking.

The reserved set guarantees each combined source at least one citation slot (its single best-scoring chunk) before any slots are filled by pure score; the remaining slots are then filled by score across all sources, and the final combined selection is re-sorted by score and truncated to k before being handed to context assembly. The floor is deliberately minimal — it guarantees representation, not proportional representation — since guaranteeing more than one slot per source would, for a query combining many sources, starve total capacity for genuinely more relevant chunks from whichever source is actually most on-topic.

D. Context Assembly

Selected chunks are assembled into the generator’s context under two separate character budgets: a total-context budget (12,000 characters, or 10,000 for an all-GitHub query, since code chunks are denser and the generator benefits from more of them fitting) and a per-chunk budget (1,600 characters, or 1,200 for GitHub) applied to each individual chunk before it is appended. Overview chunks (Section V) are exempt from the per-chunk budget and are always included in full. Chunks are appended in retrieval order, each tagged with a citation marker ([S1], [S2], ...); if a chunk’s full text would exceed the remaining total budget, it is truncated to fit and still cited, but once the budget is fully exhausted, any further chunks are dropped from the context and from the returned citation list entirely, rather than being cited without their text actually influencing the answer. GitHub chunks additionally carry a direct source link into the citation object built from the owner, repo, branch, file path, and start/end line numbers (a `github.com/.../blob/...#L{start}-{end}` permalink), and video chunks carry an [MM:SS–MM:SS] timestamp label, both attached to the structured Citation objects returned to the client, not only rendered into the prompt text.

VII. Confidence Scoring and Answer Modes

SAGE offers two answer modes. Grounded Mode is constrained to refuse when it judges its own retrieved evidence insufficient; Exploratory Mode always attempts an answer (at a higher generation temperature, 0.3 versus 0.1) and is intended for brainstorming over the ingested sources rather than fact lookup. The decision of whether Grounded Mode should refuse is made entirely from retrieval evidence, before any generation call, using a confidence score

$$\kappa = \text{clip}_{[0,1]}(0.65b + 0.35\bar{a} + 0.025\min(n, 5)), \quad (2)$$

rounded to two decimals, where b is the top-ranked chunk’s blended score (Equation 1), $\bar{a}$ is the mean blended score of the top three retrieved chunks, and n is the number of chunks actually retrieved. The last term is a small coverage bonus: retrieving five or more reasonably-ranked chunks adds up to 0.125 to the score, reflecting that several chunks in rough agreement are weaker-but-real evidence even if no single one is a very strong match.

Grounded Mode refuses outright if no chunks were retrieved at all, or if $\kappa < 0.28$ — with one deliberate exception. Chunks originating from a video source (either directly, or via a video overview chunk) are exempt from this hard refusal threshold; a low-confidence video answer is still generated, but is appended with an explicit warning (“Low confidence but transcript evidence exists. Verify citations.” when $\kappa < 0.45$, versus a plainer weak-match warning for non-video sources at the same threshold). This exemption reflects that automatic transcripts are noisier and less lexically clean than crawled or authored text, so a strict same threshold would refuse a disproportionate share of otherwise-answerable video questions; we treat the choice as a practical calibration rather than a formally justified one, and flag it as a limitation in Section X.

A separate, independent heuristic computes the user-facing confidence label (“High”/“Medium”/“Low”) and a short reason string; it is not simply a bucketing of κ . Vague or very short queries (two words or fewer, or an exact match against a small set of generic terms such as “fees,” “contact,” “summary”) are forced to at most a “Medium” label regardless of κ , on the reasoning that a vague query’s apparent confidence is not trustworthy even when retrieval happens to score well; fewer than three retrieved chunks likewise caps the label; and a strong-match shortcut (top semantic score ≥ 0.75 and top blended score ≥ 0.7) forces a “High” label before falling through to plain thresholding of κ at 0.6/0.4. This label is surfaced to the user alongside its short reason, and is deliberately decoupled from the refusal gate in Equation 2, since we found during development that a single number could not serve both purposes — gating whether to answer at all, and explaining to a user why an answer that was given should or should not be trusted — without one purpose degrading the other.

TABLE III

Worked example: five retrieved candidates before reranking

Chunk (source)	s (semantic)	ℓ (lexical)	score (Eq. 1)
c_1 (website)	0.81	0.20	0.6758
c_2 (website)	0.77	0.60	0.7326
c_3 (website)	0.74	0.40	0.6652
c_4 (repo)	0.68	0.80	0.7064
c_5 (repo)	0.55	0.20	0.4730

A. Worked Example

To make Equations 1–2 and Algorithm 1 concrete, consider a query that explicitly combines a website source and a GitHub repository source ($n = 2$), asking “how does the reranker weight lexical matches.” With $k = 6$, the widened candidate pool is $2k \cdot n = 24$ (below the cap of 40). Suppose retrieval returns, among others, five candidates with the semantic and lexical components shown in Table III.

Pure score ranking would order these $c_2(0.7326) > c_4(0.7064) > c_1(0.6758) > c_3(0.6652) > c_5(0.4730)$. If k were 2 and no diversity floor existed, the top-2 cutoff would keep only c_2 and c_4 — which, in this small example, still happens to include one chunk per source, but only because c_4 narrowly outscores c_1 and c_3 . Had c_4 ’s lexical term recall been slightly lower (say $\ell = 0.5$, giving a blended score of 0.6404), it would fall below all three website chunks and the repository would receive zero citations from an explicitly combined two-source query — exactly the silent-starvation failure described in Section VI. Algorithm 1 prevents this unconditionally rather than by chance: it reserves c_2 (best website chunk) and c_4 (best repository chunk) first, regardless of their relative scores, before backfilling any remaining slots by pure score.

Given the five scores above as the final top-5 selection ($n = 5$ chunks retrieved, top three being c_2, c_4, c_1 after reranking), Equation 2 evaluates as $b = 0.7326$, $\bar{a} = (0.7326 + 0.7064 + 0.6758)/3 = 0.7049$, and coverage bonus $= 0.025 \cdot \min(5, 5) = 0.125$, giving

$$\kappa = \text{clip}_{[0,1]}(0.65 \cdot 0.7326 + 0.35 \cdot 0.7049 + 0.125) = 0.85,$$

comfortably above the 0.28 refusal threshold and the 0.6 “High”-label threshold, so this query would be answered in Grounded Mode with a “High” confidence label and citations to both the website and the repository, in that retrieval order.

VIII. Query Rewriting, Streaming, and Memory

Query rewriting. A question is rewritten before embedding when it is very short (five words or fewer) or matches a small set of context-dependent terms (“fees,” “contact,” “about,” “deadline,” “eligibility,” “cost,” “price,” “summary”) that are usually only meaningful given prior conversation turns. Rewriting is a single temperature-0, 80-token-capped generation call that folds in up to the last

three turns of conversation history to resolve pronouns and elided context (for example, “what about the second one” following an earlier answer that listed several items); if this call times out, SAGE silently falls back to the original, un-rewritten question rather than failing the request.

Streaming. The streaming endpoint (`/qa/ask/stream`) shares its entire retrieval, reranking, diversity-floor, and confidence pipeline with the synchronous endpoint — both call the same internal retrieval-and-context-building routine, so the two surfaces cannot silently diverge in what evidence they retrieve or how they score it. The stream first emits one meta event carrying everything except the answer text itself (confidence, citations, retrieved-source list, and a follow-up question if one applies), then a sequence of delta events as the generator’s tokens arrive, and finally a done event with the fully assembled answer text and any resolved warnings. If generation times out mid-stream, SAGE still emits a done event containing the partial answer produced so far plus an appended error notice, rather than truncating the stream silently.

Memory. Conversation history is threaded through both query rewriting and the follow-up-question logic used on refusal (a small keyword match against the failed question — “summar-,” a who/speaker pattern, a when/timestamp pattern — selects a more specific clarifying question than a generic “can you be more specific” fallback). Query-history persistence to Supabase is optional (disabled by default) and, when enabled, is executed on a dedicated single-worker thread pool with a bounded 2-second timeout, so a slow or failing history write can only ever cost the user a “could not save query history” warning, never a delayed or failed answer.

A. Performance Instrumentation

Every request through either the synchronous or streaming endpoint emits one structured log line recording five timing components measured with a monotonic performance counter: retrieval time, rerank time (including the diversity floor), generation time, history-write time, and total request time. Because both endpoints share the same underlying retrieval-and-context routine (Section VIII), these five components are directly comparable across the synchronous and streaming surfaces — a regression in, say, rerank time is visible identically regardless of which endpoint a client used. We treat this as observational instrumentation rather than a benchmark: SAGE does not currently aggregate these lines into a latency percentile report, and we do not claim a quantitative latency result in this paper: doing so honestly would require dataset-scale traffic, which a manually-tested prototype does not have. We list its presence here because it is the concrete mechanism by which such a benchmark could be produced without further instrumentation work, and because per-request timing decomposition of this kind is, in our reading of the RAG systems literature, reported less consistently than retrieval-quality metrics.

IX. Security Posture (Brief)

Two mitigations are worth a brief, factual note, though security is not this paper’s subject. First, because website ingestion fetches arbitrary user-submitted URLs, SAGE resolves each URL’s hostname via DNS and rejects private, loopback, link-local, reserved, and multicast address ranges, plus a small blocklist of cloud metadata hostnames — consistent with standard SSRF guidance [28] — and, notably, re-applies this check inside the crawler’s own request-routing hook on every followed redirect during a multi-page crawl, not only on the entry URL, closing the redirect-to-internal-address bypass that a single upfront check would miss. Second, because retrieved chunk text is untrusted, attacker-influenced content by construction (it may originate from any public website a user chooses to ingest), every system prompt SAGE sends to the generator includes an explicit instruction that retrieved context is data to be quoted and reasoned about, never instructions to be obeyed, as a mitigation against indirect prompt injection [29].

X. Validation and Limitations

Test suite as evidence. SAGE’s backend test suite comprises 95 test functions across 12 files, covering: chunk ordering and page-number preservation; 28 dedicated cases for GitHub URL parsing and repository-overview generation (including determinism and language-detection checks); an end-to-end integration suite exercising the live API surface (website crawl success/failure/timeout/depth-limit paths, PDF upload validation, grounded-refusal and citation behavior, malformed-request handling); reranker and confidence formula sanity checks; source-overview metadata shape and per-modality presence at ingestion; URL normalization and deduplication; vector-store metadata upsert edge cases (Chroma rejects None-valued metadata, which the store must strip before upsert); and video-specific coverage including timestamp preservation, transcript-unavailable handling, and one live end-to-end YouTube-transcript fetch. We report this as real, existing validation evidence rather than an evaluation benchmark; SAGE does not claim a retrieval-accuracy or answer-quality metric against a labeled dataset, which is itself a limitation we state plainly below.

The diversity-floor discovery. The clearest piece of concrete evidence for the diversity floor’s necessity is its own discovery history: it was added only after manual combined-source testing surfaced answers that cited only one of two explicitly selected sources despite both containing genuinely relevant content, i.e., exactly the failure mode described in Section VI. We consider this a useful data point for RAG system builders more generally — a purely score-ranked reranker is not obviously wrong in isolation, and the defect is only visible once a system supports genuinely combined multi-source querying and is tested against it directly, rather than assumed correct because it works well in the single-source case.

Limitations. We state these plainly rather than omit them. PDF extraction has no OCR path and fails cleanly, rather than partially, on scanned documents. The video-ingestion fallback from missing-transcript to Whisper transcription requires an explicit user action rather than happening automatically, which is a usability gap rather than a technical one. The confidence formula (Equation 2) and its refusal threshold, and the separate label-bucketing heuristic, were tuned by manual inspection during development rather than fit or validated against a labeled confidence-calibration dataset; we do not claim they generalize as stated to corpora unlike the ones used during development. The background job tracker for asynchronous ingestion (GitHub and video) is an in-process, non-persistent structure with no expiry, so job status is lost on a backend restart — acceptable for a single-instance prototype, not for a multi-instance production deployment. Website recrawling always deletes and fully re-embeds an entire knowledgebase rather than diffing old and new content (Section IV), which is simple and correct but wastes embedding computation on pages that did not change between crawls; a content-hash-based incremental recrawl is a natural follow-on improvement. Finally, the Chrome extension client, despite offering streaming, multi-turn memory, and multi-source combination, currently supports website sources only; full four-modality parity exists in the React web client alone.

XI. Conclusion

SAGE demonstrates that a single ingest-chunk-embed-index-retrieve-rerank-generate contract can be shared across four structurally unrelated source types without collapsing into a lowest-common-denominator design, provided the places where modalities genuinely differ — overview-chunk construction, context character budgets, and per-source citation fairness — are made explicit, modality-aware design points rather than implicit assumptions. Its three most concrete contributions are a deterministic, non-generative overview-chunk mechanism that makes broad questions retrievable without a second LLM call; a cheap hybrid reranking formula paired with a diversity floor that was added specifically because pure score-ranking silently starved one source’s evidence in real combined-source testing; and a confidence heuristic that gates refusal from retrieval evidence alone, with an explicit, narrow exemption for noisier video transcripts. We hope the diversity-floor discovery in particular is useful to other builders of multi-source RAG systems as a concrete failure mode to test for directly, rather than assume away.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2005.11401.
- [2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. Yih, “Dense passage retrieval for open-domain question answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781. arXiv:2004.04906.
- [3] G. Izacard and E. Grave, “Leveraging passage retrieval with generative models for open domain question answering,” in *Proc. EACL*, 2021. arXiv:2007.01282.
- [4] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2023. arXiv:2312.10997.
- [5] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, 2019. arXiv:1908.10084.
- [6] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *NeurIPS*, 2020. arXiv:2002.10957.
- [7] W. Wang, H. Bao, S. Huang, L. Dong, and F. Wei, “MiniLMv2: Multi-head self-attention relation distillation for compressing pretrained transformers,” 2020. arXiv:2012.15828.
- [8] Sentence-Transformers, “all-MiniLM-L6-v2 model card,” Hugging Face. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
- [9] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [10] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” 2019. arXiv:1901.04085.
- [11] O. Khattab and M. Zaharia, “ColBERT: Efficient and effective passage search via contextualized late interaction over BERT,” in *Proc. 43rd Int. ACM SIGIR Conf.*, 2020. arXiv:2004.12832.
- [12] L. Gao, Z. Dai, and J. Callan, “Complement lexical retrieval model with semantic residual embeddings,” 2020. arXiv:2004.13969.
- [13] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods,” in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759.
- [14] A. Jimeno-Yepes, Y. You, J. Milczek, S. Laverde, and R. Li, “Financial report chunking for effective retrieval augmented generation,” 2024. arXiv:2402.05131.
- [15] M. Günther, I. Mohr, D. J. Williams, B. Wang, and H. Xiao, “Late chunking: Contextual chunk embeddings using long-context embedding models,” 2024. arXiv:2409.04701.
- [16] C. Merola and J. Singh, “Reconstructing context: Evaluating advanced chunking strategies for retrieval-augmented generation,” in *2nd Workshop on Knowledge-Enhanced Information Retrieval, ECIR*, 2025. arXiv:2504.19754.
- [17] J. Lei, L. Yu, M. Bansal, and T. L. Berg, “TVQA: Localized, compositional video question answering,” in *Proc. EMNLP*, 2018. arXiv:1809.01696.
- [18] C. Zhang, T. Lu, M. M. Islam, Z. Wang, S. Yu, M. Bansal, and G. Bertasius, “A simple LLM framework for long-range video question-answering,” in *Proc. EMNLP*, 2024. arXiv:2312.17235.
- [19] H. Husain, H.-H. Wu, T. Gazit, M. Allamanis, and M. Brockschmidt, “CodeSearchNet challenge: Evaluating the state of semantic code search,” 2019. arXiv:1909.09436.
- [20] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “RepoCoder: Repository-level code completion through iterative retrieval and generation,” in *Proc. EMNLP*, 2023. arXiv:2303.12570.
- [21] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma, “Repoformer: Selective retrieval for repository-level code completion,” in *Proc. ICML*, 2024. arXiv:2403.10059.
- [22] D. S. Sachan, S. Reddy, W. Hamilton, C. Dyer, and D. Yogatama, “End-to-end training of multi-document reader and retriever for open-domain question answering,” in *NeurIPS*, 2021. arXiv:2106.05346.
- [23] A. Kamath, R. Jia, and P. Liang, “Selective question answering under domain shift,” in *Proc. ACL*, 2020. arXiv:2006.09462.
- [24] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, S. Johnston, S. El-Showk, A. Jones, N. Elhage, T. Hume, A. Chen, Y. Bai, S. Bowman, S. Fort, D. Ganguli,

- D. Hernandez, J. Jacobson, J. Kernion, S. Kravec, L. Lovitt, K. Ndousse, C. Olsson, S. Ringer, D. Amodei, T. Brown, J. Clark, N. Joseph, B. Mann, S. McCandlish, C. Olah, and J. Kaplan, “Language models (mostly) know what they know,” 2022. arXiv:2207.05221.
- [25] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, “Survey of hallucination in natural language generation,” *ACM Computing Surveys*, vol. 55, no. 12, art. 248, 2023.
- [26] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, “Retrieval augmentation reduces hallucination in conversation,” in *Findings of EMNLP*, 2021, pp. 3784–3803. arXiv:2104.07567.
- [27] P.-S. Huang, X. He, J. Gao, L. Deng, A. Acero, and L. Heck, “Learning deep structured semantic models for web search using clickthrough data,” in *Proc. 22nd ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2013, pp. 2333–2338.
- [28] OWASP Foundation, “A10:2021 – Server-Side Request Forgery (SSRF),” OWASP Top 10:2021. [Online]. Available: [https://owasp.org/Top10/2021/A10_2021-Server-Side_Request_Forgery_\(SSRF\)/](https://owasp.org/Top10/2021/A10_2021-Server-Side_Request_Forgery_(SSRF)/)
- [29] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection,” in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. arXiv:2302.12173.
- [30] Google, “NotebookLM: How to try Google’s experimental AI-first notebook,” Google Blog, 2023. [Online]. Available: <https://blog.google/technology/ai/notebooklm-google-ai/>
- [31] LangChain, Inc., “LangChain documentation.” [Online]. Available: <https://docs.langchain.com/>
- [32] LlamaIndex, “LlamaIndex documentation.” [Online]. Available: <https://developers.llamaindex.ai/python/framework/>